

Space Station Odyssey

A Text Adventure Game

Group Name: 53 Studio

Author: Alex Rodríguez Illán

Author: Enrique Robles Puentes

Author: Juan Félix Noblejas Torroba

Exam: Metodi Avanzati di Programmazione (track M-Z)

Academic Year 2025/2026

1. Adventure Description

Space Station Odyssey is a text-based adventure game set aboard a damaged deep-space research station. The player wakes up from cryogenic suspension to find that the life-support system is failing. With oxygen running out, they must navigate through seven interconnected sectors, solve two logic puzzles, collect tools, and reach the Escape Module before time runs out.

Narrative Premise

You wake up alone aboard the SSO-7 research station. All systems are failing. The station is silent except for the hum of failing engines and the countdown of your remaining oxygen. You have 100 seconds. Find the Escape Module or die trying.

Map Layout - 7 Sectors:

- Cryogenics: Starting room. A security Keycard is available here.
- Central Hallway: Hub connecting all areas. Access to Engineering is locked behind a puzzle.
- Control Room: An Oxygen Tank can be found here. The database terminal reveals ship logs.
- Laboratory: A math puzzle locks an electronic panel hiding an emergency Hammer.
- Engineering: A Wrench is locked inside a glass display case. Break it with the Hammer.
- Communications: A working radio terminal. Use 'hack' to receive a distress message.
- Escape Module: Final destination. Use the Wrench to fix the launch console and win.

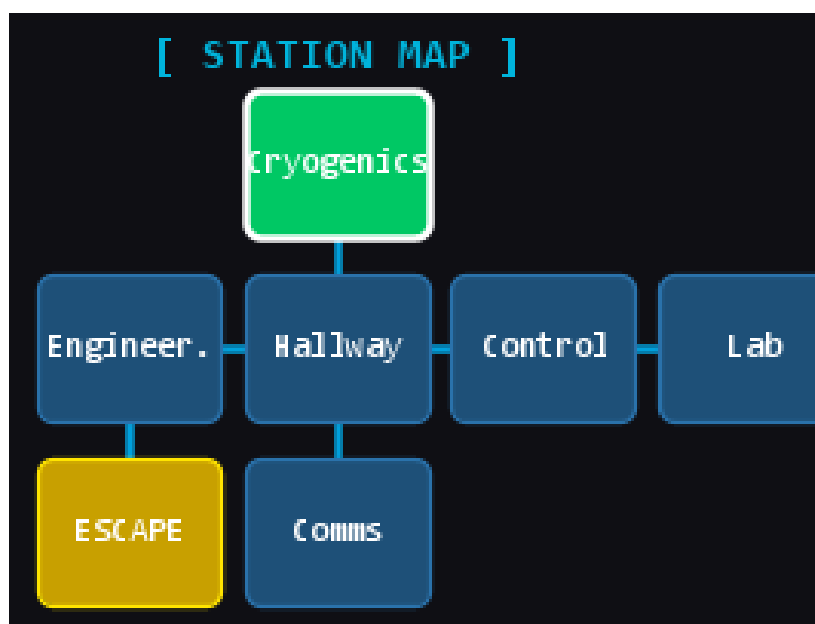


Figure 1: Space Station Odyssey - Room Map

Win Condition

Reach the Escape Module while carrying the Wrench and type 'use wrench'. The final score (remaining oxygen) is saved to the database.

Lose Condition

Oxygen reaches zero. A Game Over screen is displayed.

Puzzles

- Engineering Door: Prompt: 'Look for the prime suspects'. Answer: 2357 (first four prime numbers).
- Laboratory Panel: Prompt: 'Number of planets multiplied by months with 31 days'. Answer: 56 ($8 * 7$).

Available Commands

- go [n/s/e/w] -- Move in a direction (also: north, south, east, west)
- look / examine -- Describe the current room and list its objects
- take [object] -- Pick up an object from the room
- use [object] -- Use an item from the inventory
- break [object] -- Break something using a tool
- solve [answer] -- Submit an answer to the active puzzle
- inv -- Show the player inventory
- hack -- Interact with a network or database terminal
- save / load -- Persist and restore the game session
- help -- Display available commands

2. Design

The project follows a Model-View-Controller pattern adapted for a Swing desktop application. Design choices were guided by core OOP principles: encapsulation, inheritance, abstraction, and composition.

2.1 Package Structure

The codebase is organized into 7 packages under `di.uniba.map.b.adventure`:

- core: `GameEngine.java` (game logic orchestrator), `GameState.java` (serializable state).
- entities: `ObjetoJuego` (abstract), `Item`, `Herramienta`, `Consumible`, `Stanza`, `Contenitore`, `Direction`, `Usable`.
- parser: `Parser.java` -- interprets raw player text input into structured commands.
- gui: `MainWindow.java`, `MapPanel.java`, `GameOverWindow.java`.
- db: `DBManager.java` -- JDBC access to the H2 embedded database.
- io: `SaveLoadManager.java` -- Java object serialization for save/load.
- net: `NetworkTerminal.java` -- TCP socket server/client for the hack command.
- threads: `OxygenTimer.java` -- Countdown thread with pause/resume support.

2.2 Class Responsibilities

GameEngine

The central controller. Holds a `GameState`, receives text input from the GUI via `procesarComando()`, parses it through `Parser`, and dispatches to private methods. It implements `OxygenTimer.OxygenListener` and exposes the inner `EngineListener` interface to decouple from the GUI.

GameState

A plain serializable DTO holding the full mutable state: current room, inventory, remaining oxygen, room list, and three boolean puzzle flags (`vitrinaRota`, `puzzleLabResuelto`, `ingDoorUnlocked`).

Parser

Uses a regular expression to split player input into a verb and optional argument. Maps verbs to `ComandoType` enum values (`IR`, `TOMAR`, `USAR`, `ROMPER`, `RESOLVER`, `HACKEAR`, etc.).

Stanza (Room)

Represents a game location with an ID, display name, description, a HashMap of exits (Direction to Stanza), and a Contenaire of items in the room.

Contenaire

A generic container backed by LinkedHashSet. Used for both the player inventory and room item lists. filtrarPorNombre uses Java streams for case-insensitive name matching.

ObjetoJuego Hierarchy

ObjetoJuego is an abstract class implementing Usable. Three concrete subclasses: Item (generic pickable), Herramienta (reusable tool), Consumible (single-use with consumed flag).

MainWindow

JFrame implementing GameEngine.EngineListener. Builds the sci-fi Swing UI and delegates all game logic to GameEngine.

MapPanel

Custom JPanel using Graphics2D to draw the station map. Rooms render as rounded rectangles with fog-of-war: unvisited rooms show as '???'.

3. Class Diagram

The diagram below shows the main entities and their relationships: inheritance from ObjetoJuego, interface implementation of Usable, generics in Contenitore, and composition between Stanza, GameState, and GameEngine.

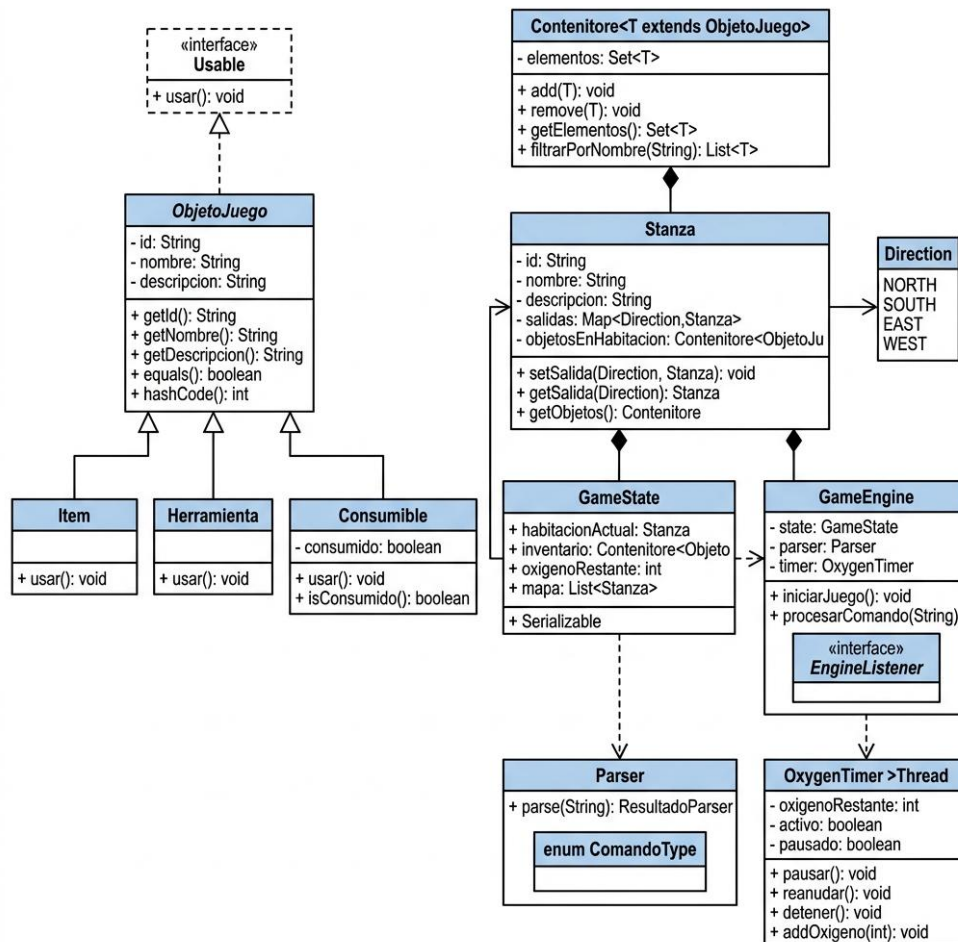


Figure 2: UML Class Diagram - Core Entity Model

Key OOP principles applied:

- Abstraction: ObjetoJuego defines the common interface for all game objects.
- Inheritance: Item, Herramienta, Consumible extend ObjetoJuego without duplicating fields.
- Interface: The Usable interface decouples the concept of using an object from its implementation.
- Generics: Contenitore is reused type-safely for both inventory and room contents.
- Composition: Stanza has-a Contenitore; GameEngine has-a GameState, Parser, OxygenTimer.
- Encapsulation: All fields are private; state accessed only through public accessor methods.

4. Algebraic Specification

Non-axiomatic algebraic specification for `Contenitore` -- the generic collection used as both player inventory and room item storage. Backed by `LinkedHashSet`: holds distinct elements with insertion-order iteration.

4.1 Sorts

- `Contenitore` -- the collection type being specified.
- `Element` (`T` extends `ObjetoJuego`) -- the element type stored.
- `Boolean` -- standard boolean sort.
- `List[Element]` -- list of elements (filter result type).
- `String` -- standard string sort.

4.2 Operations

- `new : -> Contenitore` (creates an empty container)
- `add : Contenitore x Element -> Contenitore` (adds element; no-op if already present)
- `remove : Contenitore x Element -> Contenitore` (removes element; no-op if absent)
- `getElementos : Contenitore -> Set[Element]` (returns all elements)
- `filtrarPorNombre : Contenitore x String -> List[Element]` (returns elements whose name contains the string)

4.3 Properties

Let `c = new`, `e1` be an `Element`, `s` be a `String`:

- `getElementos(new) = {}`
- `getElementos(add(c, e1)) = getElementos(c) UNION {e1}`
- `add(add(c, e1), e1) = add(c, e1)` -- idempotency
- `getElementos(remove(add(c, e1), e1)) = getElementos(c)` -- add then remove restores original
- `remove(c, e1) = c` if `e1 NOT IN getElementos(c)` -- removing absent element is no-op
- `filtrarPorNombre(new, s) = []`
- `filtrarPorNombre(add(c,e1), s) = [e1] + filtrarPorNombre(c,s)` if `e1.nombre.lower` contains `s.lower`
- `filtrarPorNombre(add(c,e1), s) = filtrarPorNombre(c,s)` otherwise

5. Implementation Details

5.1 Generic Programming

Generics are used in `Contenitore`. The bounded type parameter `T` extends `ObjetoJuego` ensures type safety and allows reuse for inventory and room contents:

```
public class Contenitore<T extends ObjetoJuego> implements Serializable {
    private Set<T> elementos = new LinkedHashSet<>();
    public void add(T elemento) { elementos.add(elemento); }
    public List<T> filtrarPorNombre(String filtro) {
        return elementos.stream()
            .filter(e
e.getNombre().toLowerCase().contains(filtro.toLowerCase()))
            .collect(Collectors.toList());
    }
}
```

5.2 Files

File I/O is used for the save/load feature. `SaveLoadManager` uses Java object serialization to persist the entire `GameState` to a binary file (`savegame.dat`). All related objects implement `Serializable` with an explicit `serialVersionUID`.

```
public static boolean guardar(GameState state) {
    try (ObjectOutputStream out =
        new ObjectOutputStream(new FileOutputStream(SAVE_FILE))) {
        out.writeObject(state);
        return true;
    } catch (IOException e) { return false; }
}
```

5.3 Database (JDBC)

An embedded H2 database is used via JDBC for storing terminal logs and persisting the final score. `DBManager` uses `PreparedStatement` for safe parameterized queries.

```
private static final String URL =
public static void guardarPuntuacion(String nombre, int score) {
    String sql =
    try (PreparedStatement ps = getConnection().prepareStatement(sql)) {
```

```

        ps.setString(1, nombre);
        ps.setInt(2, score);
        ps.executeUpdate();
    } catch (SQLException e) { ... }
}

```

5.4 Lambda Expressions, Streams and Pipelines

Lambda expressions and Stream API are used in:

- `Contenitore.filtrarPorNombre()`: stream pipeline with filter lambda and collect terminal.
- `MainWindow` button action listeners: lambdas replace anonymous inner classes.
- `NetworkTerminal` server thread: defined as a lambda passed to `Thread` constructor.
- `SwingUtilities.invokeLater()`: lambdas schedule GUI updates on the Event Dispatch Thread.

```

// Stream + lambda pipeline in Contenitore.java
return elementos.stream()
    .filter(e -> e.getNombre().toLowerCase().contains(filtro.toLowerCase()))
    .collect(Collectors.toList());

```

5.5 Swing

The GUI is built entirely with Java Swing. `MainWindow` extends `JFrame` and implements `GameEngine.EngineListener`, decoupling view from model. Key components:

- `JFrame` with `BorderLayout` as the main container.
- `JTextArea` (read-only) for the narrative log, inside `JScrollPane`.
- `JProgressBar` for oxygen countdown, dynamically changing color (green > orange > red).
- `JList` with `DefaultListModel` for the inventory panel.
- Custom `JPanel` (`MapPanel`) with `paintComponent` overridden for the schematic map.
- `JLayeredPane` with `OverlayLayout` for the pause screen overlay.
- `GridLayout` compass pad and colored quick-action buttons.
- `JOptionPane` for prompting object names in Take and Use actions.
- `SwingUtilities.invokeLater()` for all GUI updates from background threads.

5.6 Threads and Concurrent Programming

The oxygen countdown runs on a dedicated background thread (`OxygenTimer` extends `Thread`). Thread safety mechanisms:

- Pause/Resume: synchronized block on a monitor object (lockPausa). Pause calls wait(); resume calls notifyAll(). The pausado flag is volatile.
- Stop: activo volatile flag set to false; notifyAll() unblocks any waiting thread.
- EDT safety: all GUI updates from the timer use SwingUtilities.invokeLater().
- Daemon thread: NetworkTerminal server is a daemon so it does not prevent JVM shutdown.

```
// Pause mechanism in OxygenTimer.java
synchronized (lockPausa) {
    while (pausado && activo) {
        lockPausa.wait();
    }
}
```

5.7 Sockets

The hack command in the Communications room uses Java TCP sockets. NetworkTerminal starts a ServerSocket on port 5555 as a daemon thread at game startup. When the player hacks Communications, a client socket connects to localhost:5555 and reads the response line.

```
public static String getMessageFromServer() {
    try (Socket socket = new Socket(
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()))) {
        return in.readLine();
    } catch (IOException e) {
        return
    }
}
```